

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

ОТЧЕТ по лабораторной работе
по дисциплине «Численные методы»
Тема: Решение нелинейных уравнений
Вариант № 11

Выполнил:
Студент 3 курса, 7 группы
Стаселько Илья Юрьевич

Преподаватель:
Горбачева Юлия Николаевна

Минск 2026

1 Отделение корня

1.1 Графическое отделение корня

Для локализации корня исходное уравнение $f(x) = 0$ было представлено в виде равенства двух функций: $\sqrt{x+2} = 2\cos(x)$. Был построен график левой и правой частей уравнения.

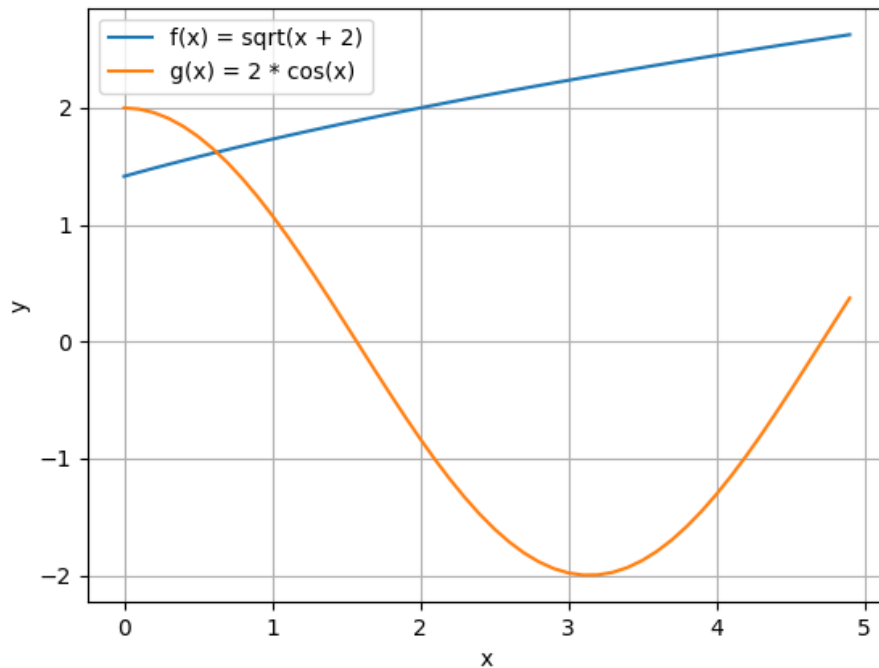


Рис. 1: Графическое отделение корня

Исходя из графиков, точка пересечения функций (корень уравнения) находится на отрезке $[0.5; 1]$.

1.2 Алгоритм метода половинного деления

Метод половинного деления применяется для сужения отрезка локализации корня до длины $2\varepsilon = 0.2$ (так как заданная точность отделения $\varepsilon = 0.1$). Алгоритм:

1. Вычисляется середина отрезка: $x = \frac{a+b}{2}$.
2. Проверяется условие: если $f(x) \cdot f(a) < 0$, то корень лежит на отрезке $[a, x]$, и полагается $b = x$. Иначе корень лежит на отрезке $[x, b]$, и полагается $a = x$.
3. Процесс повторяется, пока $|b - a| > 2\varepsilon$.

Сводные данные по результатам работы метода представлены в таблице 1.

Таблица 1: Результаты метода половинного деления

k	a_k	b_k	$f(a_k)$	$f(b_k)$	$b_k - a_k$
0	0.500	0.750	-0.174026	0.194935	0.250
1	0.625	0.750	-0.001741	0.194935	0.125

В качестве отрезка отделенного корня для дальнейших вычислений принимаем суженный отрезок: $[0.625; 0.75]$.

2 Алгоритмы методов уточнения корня

Используется отрезок $[0.625, 0.75]$, начальное приближение выбрано как середина отрезка $x_0 = 0.6875$.

2.1 Метод простой итерации

Исходное уравнение приводится к каноническому виду $x = \varphi(x)$ путем введения параметра c : $\varphi(x) = x + c \cdot f(x)$. Для выполнения условия сходимости $|\varphi'(x)| < 1$, параметр c выбирается по формуле $c = -\frac{2}{M+m}$, где M и m — максимальное и минимальное значения производной $f'(x)$ на суженном отрезке. Итерационная формула:

$$x^{k+1} = \varphi(x^k) = x^k + c \cdot f(x^k)$$

2.2 Метод Ньютона

Метод основан на замене функции касательной в точке приближения. Требуется вычисления производной $f'(x)$. Итерационная формула:

$$x^{k+1} = x^k - \frac{f(x^k)}{f'(x^k)}$$

2.3 Метод секущих

Модификация метода Ньютона, где производная заменяется разностным отношением, что избавляет от необходимости вычислять производную на каждом шаге. Является двухшаговым методом. В качестве x^0 и x^1 взяты концы суженного отрезка. Итерационная формула:

$$x^{k+1} = x^k - f(x^k) \frac{x^k - x^{k-1}}{f(x^k) - f(x^{k-1})}$$

3 Проверка условий теоремы о сходимости

Необходимо проверить условия теоремы о сходимости для **метода Ньютона**.

Теорема требует выполнения условия: $2|h_0|M \leq |f'(x^0)|$, где $h_0 = -\frac{f(x_0)}{f'(x_0)}$, а $M = \max_{x \in s_0} |f''(x)|$ на отрезке $s_0 = [x^0, x^0 + 2h_0]$.

Подставляя значения из программы для $x_0 = 0.6875$:

- $h_0 = -0.05952$
- Отрезок s_0 : $[0.56847, 0.68750]$
- $M = \max_{x \in s_0} |f''(x)| = 1.62472$
- $|f'(x_0)| = 1.57421$

Проверяем неравенство:

$$2 \cdot 0.05952 \cdot 1.62472 \leq 1.57421 \implies 0.19339 \leq 1.57421$$

Вывод: Неравенство строго выполняется. Условия теоремы соблюдены, метод Ньютона на данном отрезке гарантированно сходится.

4 Сводные данные по результатам работы методов

Результаты итерационных процессов представлены в таблице 2.

Таблица 2: Сводные данные уточнения корня с точностью $\varepsilon = 10^{-10}$

Номер итерации k	Метод простой итерации		Метод Ньютона		Метод секущих	
	x^k	$ x^k - x^{k-1} $	x^k	$ x^k - x^{k-1} $	x^k	$ x^k - x^{k-1} $
0	0.6875000000	—	0.6875000000	—	0.7500000000	—
1	0.6278931585	5.96068e-02	0.6279846486	5.95153e-02	0.6250000000	1.25000e-01
2	0.6262747036	1.61845e-03	0.6261783363	1.80631e-03	0.6261065578	1.10655e-03
3	0.6261823007	9.24028e-05	0.6261766170	1.71929e-06	0.6261766605	7.01027e-05
4	0.6261769466	5.35408e-06	0.6261766170	1.55886e-12	0.6261766170	4.35192e-08
5	0.6261766361	3.10491e-07	—	—	0.6261766170	1.60793e-12
6	0.6261766181	1.80067e-08	—	—	—	—
7	0.6261766171	1.04428e-09	—	—	—	—
8	0.6261766170	6.05628e-11	—	—	—	—

Искомый корень уравнения: $x \approx 0.6261766170$.

5 Выводы

На основе проведенного сравнительного анализа работы трех алгоритмов можно сделать следующие выводы о скорости их сходимости:

1. **Метод Ньютона** продемонстрировал наивысшую скорость сходимости (квадратичную). Заданная точность $\varepsilon = 10^{-10}$ была достигнута всего за 4 итерации. Погрешность убывала стремительно.
2. **Метод секущих** показал высокую (сверхлинейную) скорость сходимости. Ему потребовалось 5 итераций. Его главное преимущество перед методом Ньютона заключается в отсутствии необходимости аналитического вычисления производной, что делает его более универсальным.
3. **Метод простой итерации** оказался самым медленным из рассмотренных (линейная сходимость). Для достижения требуемой точности потребовалось 8 итераций.

Все три метода сошлись к одному и тому же значению корня, что подтверждает правильность вычислений.

6 Листинг программы

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4
5 # Рисуем графики функций  $f(x) = \sqrt{x + 2}$  и  $g(x) = 2 * \cos(x)$ :
6
7
8 def f(x):
9     return (x + 2) ** 0.5
10
11
12 def g(x):
13     return 2 * np.cos(x)
14
15
16
17 x_values = np.arange(0, 5, 0.1)
18 f_values, g_values = [f(x) for x in x_values], [g(x) for x in x_values]
19 plt.plot(x_values, f_values, label="f(x) = sqrt(x + 2)")
20 plt.plot(x_values, g_values, label="g(x) = 2 * cos(x)")
21 plt.xlabel("x")
22 plt.ylabel("y")
23 plt.legend()
24 plt.grid(True)
25 plt.show()
26
27 # Корень уравнения лежит в отрезке [0.5; 1], исходя из графиков
28
29
30 # Применим метод половинного деления для нахождения корня уравнения  $f(x) = g(x)$ :
31
32
33 def h(x):
34     return f(x) - g(x)
35
36
37 a, b = 0.5, 1
38 eps = 0.1
39 k = 0
40 while abs(b - a) > 2 * eps:
41     x = (a + b) / 2
42     if h(x) * h(a) < 0:
43         b = x
44     else:
45         a = x
46     print(
47         f"k = {k}, a = {a}, b = {b}, x = {x}, h(a) = {h(a)}, h(b) = {h(b)},
48         h(x) = {h(x)}, |b - a| = {abs(b - a)}"
49     )
50     print("-----")
51     k += 1
52
53 # Беру отрезок [0.625; 0.75]
54
55 # Метод простой итерации:
56
```

```

57 # Целевая точность
58 eps = 1e-10
59
60
61 # Производная функции  $h(x)$  для проверки условия сходимости и поиска констант
62 def dh(x):
63     return 1 / (2 * (x + 2) ** 0.5) + 2 * np.sin(x)
64
65
66 # Находим минимум и максимум производной на суженном отрезке  $[a, b]$ 
67 # Так как функция производной монотонно возрастает на этом отрезке:
68 m = dh(a) # Минимум
69 M = dh(b) # Максимум
70
71 # Выбираем оптимальный параметр  $c$ 
72 c = -2 / (M + m)
73
74
75 # Определяем функцию  $\phi(x)$  для канонического вида  $x = \phi(x)$ 
76 def phi(x):
77     return x + c * h(x)
78
79
80 # В качестве начального приближения возьму середину отрезка  $[a, b]$ 
81 x_k = (a + b) / 2
82 k = 0
83
84 print("Метод простой итерации")
85 print(f"{'Номер':<10} | {'Метод простой итерации':^35}")
86 print(f"{'Итерации k':<10} | {'x^k':<16} | {'|x^k - x^{k-1}|':<16}")
87 print("-" * 48)
88
89 print(f"{'k':<10} | {'x_k':<16.10f} | {'-':^16}")
90
91 # Итерационный процесс
92 while True:
93     k += 1
94     x_prev = x_k
95     x_k = phi(x_prev) # Шаг итерации
96
97     diff = abs(x_k - x_prev)
98
99     # Вывод текущей итерации
100     print(f"{'k':<10} | {'x_k':<16.10f} | {'diff':<16.10e}")
101
102     # Критерий остановки
103     if diff < eps:
104         break
105
106 print("-" * 48)
107 print(f"Корень уравнения (с точностью eps = {eps}): {x_k:.10f}")
108
109
110 # Метод Ньютона:
111
112 # Вторая производная функции  $h(x)$ 
113 def d2h(x):
114     return -0.25 * (x + 2)**(-1.5) + 2 * np.cos(x)
115

```

```

116 print("\n--- Проверка условий теоремы о сходимости метода Ньютона ---")
117
118 # В качестве начального приближения возьму середину отрезка [a, b]
119 x0 = (a + b) / 2
120 f_x0 = h(x0)
121 df_x0 = dh(x0)
122
123 h0 = - f_x0 / df_x0
124
125 # Определяем границы отрезка s_0
126 s0_start = min(x0, x0 + 2 * h0)
127 s0_end = max(x0, x0 + 2 * h0)
128
129 # Максимум модуля второй производной на отрезке s_0.
130 # Поскольку d2h(x) на этом интервале монотонно убывает,
131 # максимум модуля будет на одном из концов отрезка s_0
132 M_newton = max(abs(d2h(s0_start)), abs(d2h(s0_end)))
133
134 condition_left = 2 * abs(h0) * M_newton
135 condition_right = abs(df_x0)
136
137 print(f"Начальное приближение x_0 = {x0}")
138 print(f"h_0 = -f(x_0)/f'(x_0) = {h0:.5f}")
139 print(f"Отрезок s_0: [{s0_start:.5f}, {s0_end:.5f}]")
140 print(f"M = max |f''(x)| на s_0 = {M_newton:.5f}")
141 print(f"Проверяем неравенство 2*|h_0|*M <= |f'(x_0)|:")
142 print(f"{condition_left:.5f} < {condition_right:.5f}")
143
144 if condition_left <= condition_right:
145     print("ВЫВОД: Условие теоремы выполняется. Метод Ньютона гарантированно
146           сходится!\n")
147 else:
148     print("ВЫВОД: Условие не выполняется. Возможны проблемы со сходимостью.\n")
149
150 # Реализация метода Ньютона
151 x_k_newton = x0
152 k_iter_newton = 0
153
154 print("Метод Ньютона")
155 print(f"{'Номер':<10} | {'Метод Ньютона':^35}")
156 print(f"{'итерации k':<10} | {'x^k':<16} | {'|x^k - x^{k-1}|':<16}")
157 print("-" * 48)
158
159 # Нулевая итерация
160 print(f"{'k_iter_newton':<10} | {'x_k_newton':<16.10f} | {'-':^16}")
161
162 while True:
163     k_iter_newton += 1
164     x_prev_newton = x_k_newton
165
166     # Формула метода Ньютона
167     x_k_newton = x_prev_newton - h(x_prev_newton) / dh(x_prev_newton)
168
169     diff_newton = abs(x_k_newton - x_prev_newton)
170
171     # Вывод текущей итерации
172     print(f"{'k_iter_newton':<10} | {'x_k_newton':<16.10f} | {'diff_newton':<16.10
173           e}")

```

```

173
174     # Критерий остановки с заданной точностью  $10^{-10}$ 
175     if diff_newton < eps:
176         break
177
178 print("-" * 48)
179 print(f"Корень уравнения (с точностью eps = {eps}): {x_k_newton:.10f}")
180
181
182 # Метод секущих:
183
184 # Для метода секущих нужны два начальных приближения:  $x^0$  и  $x^1$ 
185 # Возьмем концы отрезка  $[a, b]$ 
186 x_prev_sec = b # это будет  $x^0$ 
187 x_curr_sec = a # это будет  $x^1$ 
188
189 k_iter_sec = 0
190
191 print("\nМетод секущих")
192 print(f"{'Номер':<10} | {'Метод секущих':^35}")
193 print(f"{'Итерации k':<10} | {' $x^k$ ':<16} | {' $|x^k - x^{k-1}|$ ':<16}")
194 print("-" * 48)
195
196 # Вывод нулевой итерации  $k = 0$ 
197 print(f"{'k_iter_sec':<10} | {'x_prev_sec':<16.10f} | {'-':^16}")
198
199 # Вывод первой итерации  $k = 1$ 
200 k_iter_sec += 1
201 diff_sec = abs(x_curr_sec - x_prev_sec)
202 print(f"{'k_iter_sec':<10} | {'x_curr_sec':<16.10f} | {'diff_sec':<16.10e}")
203
204 # Итерационный процесс для  $k = 2, 3, \dots$ 
205 while True:
206     k_iter_sec += 1
207
208     # Значения функции в точках  $x^{k-1}$  и  $x^k$ 
209     f_prev = h(x_prev_sec)
210     f_curr = h(x_curr_sec)
211
212     # Формула метода секущих
213     x_next_sec = x_curr_sec - f_curr * (x_curr_sec - x_prev_sec) / (f_curr -
214                                     f_prev)
215
216     diff_sec = abs(x_next_sec - x_curr_sec)
217
218     # Вывод текущей итерации
219     print(f"{'k_iter_sec':<10} | {'x_next_sec':<16.10f} | {'diff_sec':<16.10e}")
220
221     # Сдвигаем точки для следующего шага
222     x_prev_sec, x_curr_sec = x_curr_sec, x_next_sec
223
224     # Критерий остановки с заданной точностью  $10^{-10}$ 
225     if diff_sec < eps:
226         break
227
228 print("-" * 48)
229 print(f"Корень уравнения (с точностью eps = {eps}): {x_curr_sec:.10f}")

```